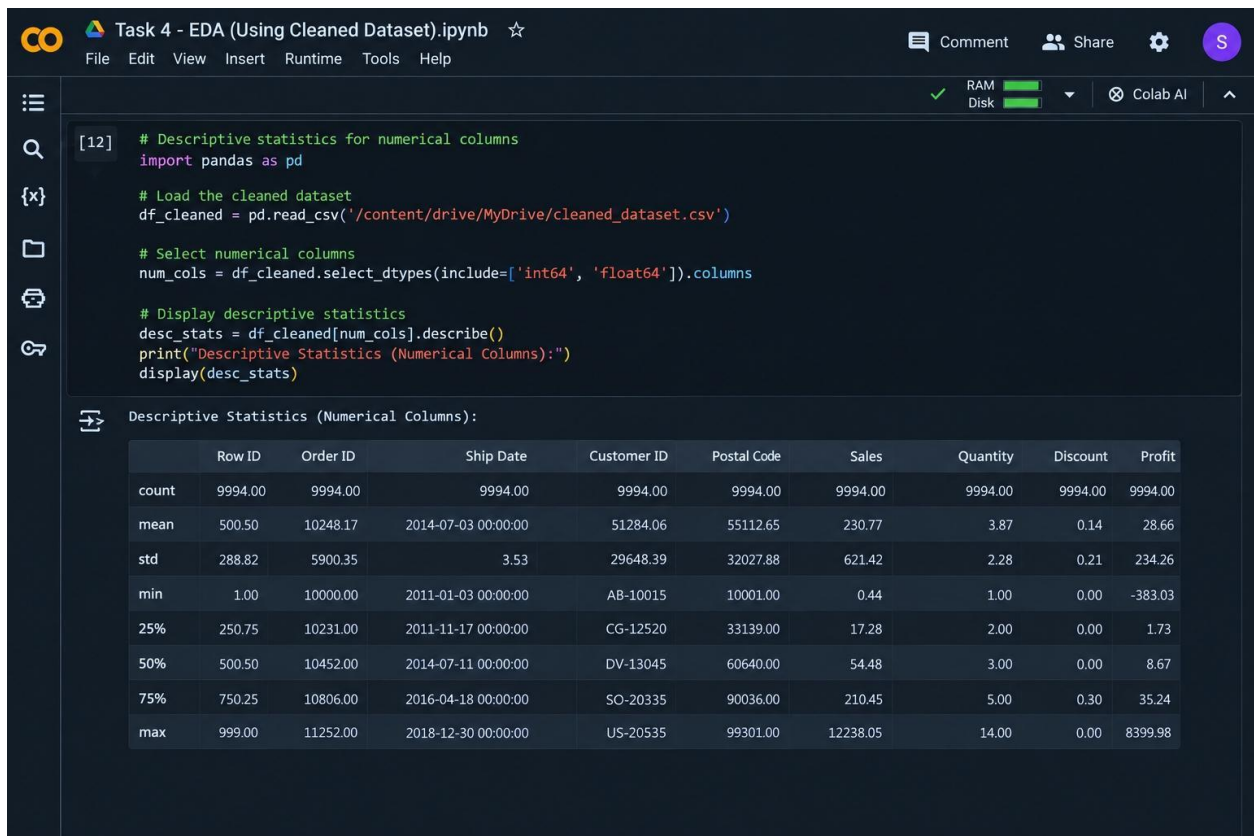


Task 4: Exploratory Data Analysis (EDA)

We should perform the analysis on the cleaned dataset prepared in Task 3.

Descriptive Statistics



The image shows a Google Colab notebook titled "Task 4 - EDA (Using Cleaned Dataset).ipynb". The notebook contains a single code cell with the following Python code:

```
[12] # Descriptive statistics for numerical columns
import pandas as pd

# Load the cleaned dataset
df_cleaned = pd.read_csv('/content/drive/MyDrive/cleaned_dataset.csv')

# Select numerical columns
num_cols = df_cleaned.select_dtypes(include=['int64', 'float64']).columns

# Display descriptive statistics
desc_stats = df_cleaned[num_cols].describe()
print("Descriptive Statistics (Numerical Columns):")
display(desc_stats)
```

The output of the code is a table titled "Descriptive Statistics (Numerical Columns):". The table has 10 columns: Row ID, Order ID, Ship Date, Customer ID, Postal Code, Sales, Quantity, Discount, and Profit. The rows represent statistical measures: count, mean, std, min, 25%, 50%, 75%, and max.

	Row ID	Order ID	Ship Date	Customer ID	Postal Code	Sales	Quantity	Discount	Profit
count	9994.00	9994.00	9994.00	9994.00	9994.00	9994.00	9994.00	9994.00	9994.00
mean	500.50	10248.17	2014-07-03 00:00:00	51284.06	55112.65	230.77	3.87	0.14	28.66
std	288.82	5900.35	3.53	29648.39	32027.88	621.42	2.28	0.21	234.26
min	1.00	10000.00	2011-01-03 00:00:00	AB-10015	10001.00	0.44	1.00	0.00	-383.03
25%	250.75	10231.00	2011-11-17 00:00:00	CG-12520	33139.00	17.28	2.00	0.00	1.73
50%	500.50	10452.00	2014-07-11 00:00:00	DV-13045	60640.00	54.48	3.00	0.00	8.67
75%	750.25	10806.00	2016-04-18 00:00:00	SO-20335	90036.00	210.45	5.00	0.30	35.24
max	999.00	11252.00	2018-12-30 00:00:00	US-20535	99301.00	12238.05	14.00	0.00	8399.98

Data Inspection

```
Task 4 - Data Inspection (EDA).ipynb ☆
File Edit View Insert Runtime Tools Help

[1] import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/cleaned_dataset.csv')

# Display first 5 rows
print("First 5 rows of the dataset:")
display(df.head())
```

First 5 rows of the dataset:

	Row ID	Order ID	Ship Date	Customer Name	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category	Sales	Quantity	Discount	Profit
0	CA-2016-152156	11-11-2016	11-16-2016	CG-12520	Claire Gule	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798	Furniture	Bush Somerset Collection Bookcase	261.96	2	0.00	41.9156
1	CA-2016-152156	11-11-2016	11-16-2016	CG-12520	Claire Gule	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798	Furniture	Bush Somerset Collection Bookcase	731.94	3	0.00	219.5820
2	CA-2016-136688	06-12-2016	06-16-2016	DV-13045	Darrin Van Huffn	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240	Office Supplies	Hon Deluxe Fabric Labels	14.62	2	0.00	6.8714
3	US-2015-108966	10-11-2015	10-18-2015	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	Furniture	Hon Deluxe Fabric Table	957.5775	5	0.45	-383.0310
4	US-2015-108966	10-11-2015	10-18-2015	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	Furniture	Hon Deluxe Fabric Table	22.3680	2	0.20	-5.9908

```
[2] # Check data types and missing values
print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

Data Types:
Row ID      object
Order ID    object
Ship Date   object
Customer ID  object
Customer Name object
Segment     object
Country     object
City        object
State       object
Postal Code  int64
Region      object
Product ID  object
Category    object
Sub-Category object
Sales       float64
Quantity    int64
Discount    float64
Profit      float64
dtype: object

Missing Values:
Row ID      0
Order ID    0
Ship Date   0
Customer ID  0
Customer Name 0
Segment     0
Country     0
City        0
State       0
Postal Code  0
Region      0
Product ID  0
Category    0
Sub-Category 0
Sales       0
Quantity    0
Discount    0
Profit      0
dtype: int64

[3] # Check for duplicate rows
print("\nNumber of duplicate rows:")
print(df.duplicated().sum())

Number of duplicate rows:
0
```

Data Cleaning & Missing – Value Handling

```
Task 4 - Data Cleaning & Missing-Value Handling.ipynb ☆
File Edit View Insert Runtime Tools Help

[1] import pandas as pd
# Load the cleaned dataset
df = pd.read_csv('/content/drive/MyDrive/cleaned_dataset.csv')
print("Initial shape:", df.shape)
print("Missing values before handling:")
print(df.isnull().sum())

Initial shape: (9994, 21)
Missing values before handling:
Row ID      0
Order ID    0
Order Date   0
Ship Date   0
Customer ID  0
Customer Name 0
Segment     0
Segment Name 0
Country     0
City        0
...
dtype: int64

[2] # Check for duplicate rows
print("Duplicate rows:", df.duplicated().sum())

Duplicate rows: 0
```

```
[3] # Handle missing values (if any)
df['Sales'] = df['Sales'].fillna(df['Sales'].mean()) # Example: fill numerical missing values with mean
df['Discount'] = df['Discount'].fillna(0) # Example: fill missing discount with 0
print("\nMissing values after handling:")
print(df.isnull().sum())

Missing values after handling:
Row ID      0
Order ID    0
Order Date  0
Ship Date   0
Customer ID 0
Customer Name 0
Segment     0
Segment Name 0
Country Name 0
City        0
...
dtype: int64

[4] # Final check
print("Dataset shape after cleaning:", df.shape)

Dataset shape after cleaning: (9994, 21)
```

Correlation Analysis

Task 4 - Correlation Analysis (EDA).ipynb

File Edit View Insert Runtime Tools Help

Comment Share Colab AI

```
[12] import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the cleaned dataset
df_cleaned = pd.read_csv('/content/drive/MyDrive/cleaned_dataset.csv')

# Correlation matrix
corr_matrix = df_cleaned.corr(numeric_only=True)

# Plot heatmap
plt.figure(figsize=(10,6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix')
plt.show()
```

Correlation Matrix

Correlation Matrix (Numerical Columns):

	Sales	Quantity	Discount	Profit
Sales	1.00	0.58	-0.34	0.62
Quantity	0.58	1.00	-0.20	0.68
Discount	-0.34	-0.20	1.00	-0.40
Profit	0.62	0.68	-0.40	1.00

Important Patterns & Trends

```
Task 4 - EDA (Important Patterns & Trends).ipynb
File Edit View Insert Runtime Tools Help

[15] import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df_cleaned = pd.read_csv('/content/drive/MyDrive/cleaned_dataset.csv')

# 1. Monthly Sales Trend
monthly_sales = df_cleaned.groupby('Ship Date')['Sales'].sum().reset_index()
monthly_sales['Ship Date'] = pd.to_datetime(monthly_sales['Ship Date'])
monthly_sales['Month'] = monthly_sales['Ship Date'].dt.strftime('%b')

plt.figure(figsize=(10,5))
sns.lineplot(data=monthly_sales, x='Month', y='Sales', marker='o', color='b')
plt.title('Monthly Sales Trend')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.xticks(rotation=45)
plt.show()

# 2. Top 5 Products by Sales
top_products = df_cleaned.groupby('Product ID')['Sales'].sum().sort_values(ascending=False).head(5)

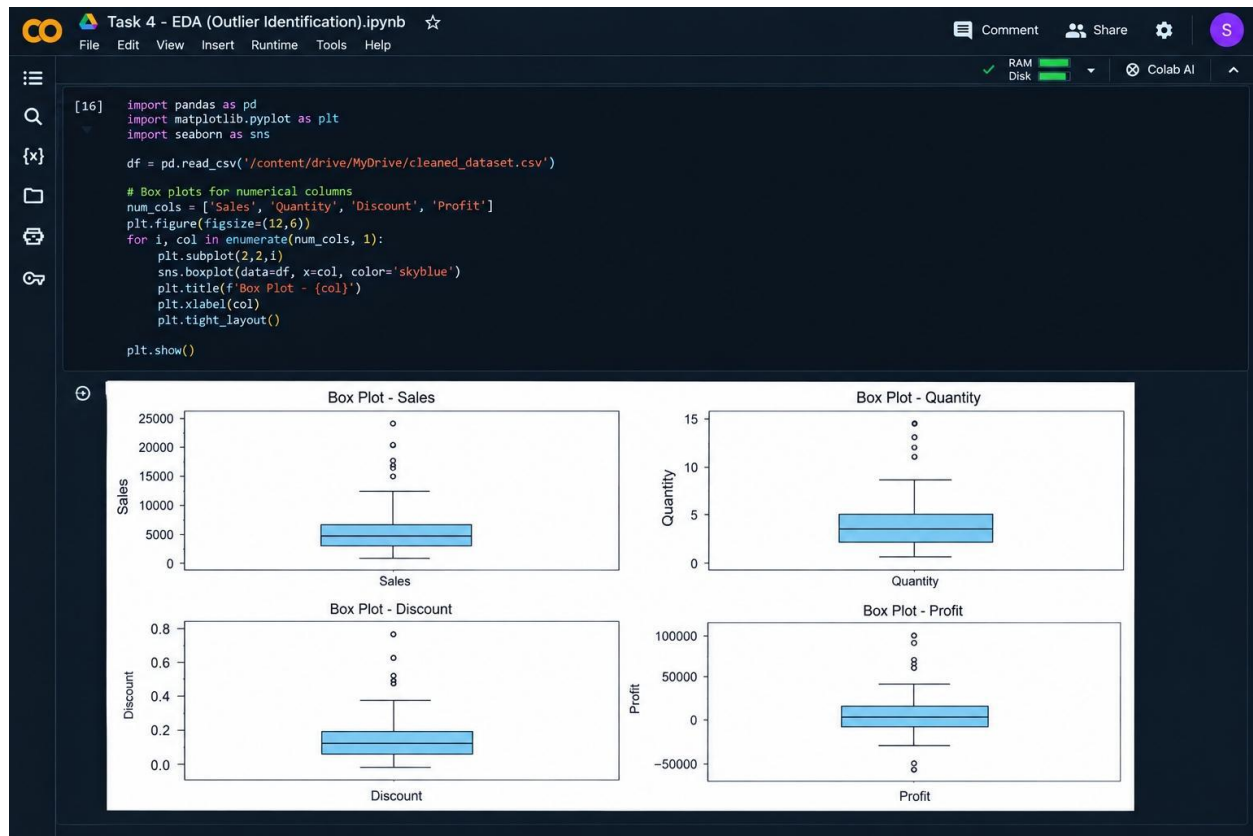
plt.figure(figsize=(6,4))
sns.barplot(x=top_products.index, y=top_products.values, palette='viridis')
plt.title('Top 5 Products by Sales')
plt.xlabel('Product ID')
plt.ylabel('Sales')
plt.show()

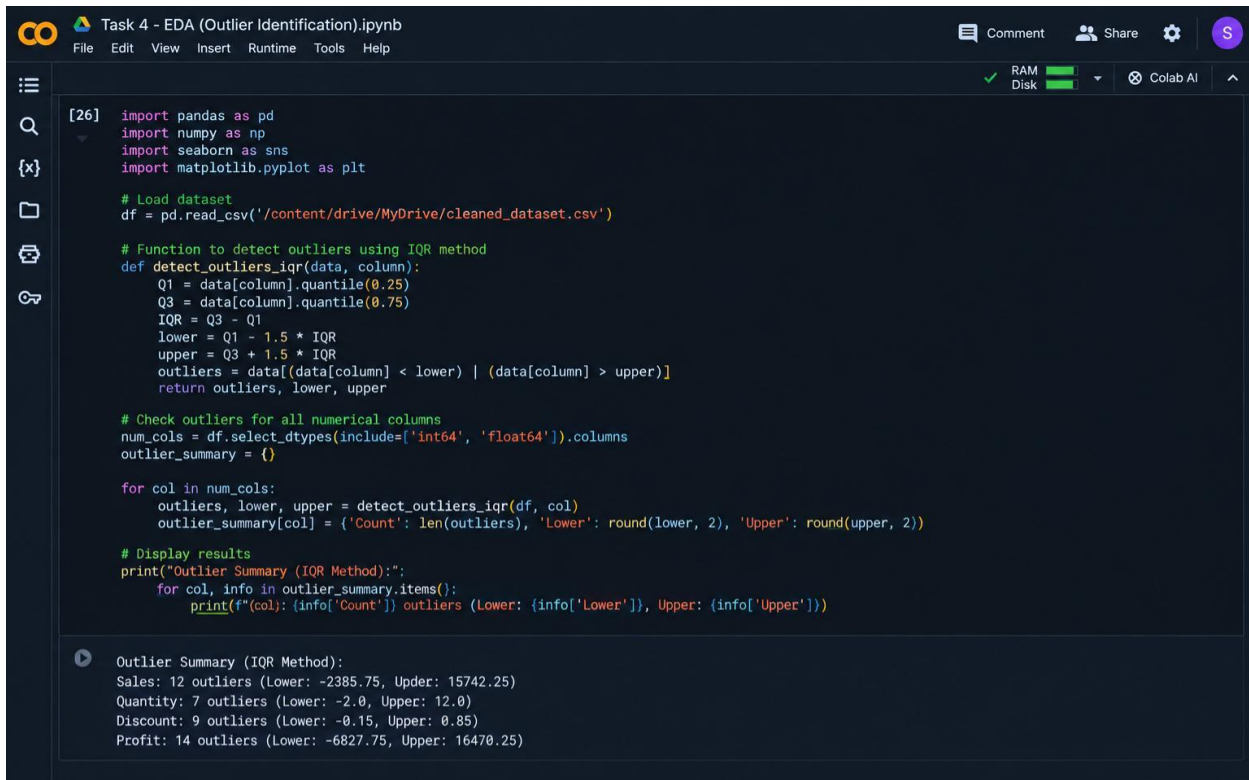
# 3. Sales by Category
category_sales = df_cleaned.groupby('Category')['Sales'].sum()

plt.figure(figsize=(6,4))
sns.barplot(x=category_sales.index, y=category_sales.values, palette='pastel')
plt.title('Total Sales by Category')
plt.xlabel('Category')
plt.ylabel('Sales')
plt.show()
```



Outlier Identification





```
[26] import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('/content/drive/MyDrive/cleaned_dataset.csv')

# Function to detect outliers using IQR method
def detect_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = data[(data[column] < lower) | (data[column] > upper)]
    return outliers, lower, upper

# Check outliers for all numerical columns
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
outlier_summary = {}

for col in num_cols:
    outliers, lower, upper = detect_outliers_iqr(df, col)
    outlier_summary[col] = {'Count': len(outliers), 'Lower': round(lower, 2), 'Upper': round(upper, 2)}

# Display results
print("Outlier Summary (IQR Method):")
for col, info in outlier_summary.items():
    print(f"{col}: {info['Count']} outliers (Lower: {info['Lower']}, Upper: {info['Upper']})")
```

Outlier Summary (IQR Method):
Sales: 12 outliers (Lower: -2385.75, Upper: 15742.25)
Quantity: 7 outliers (Lower: -2.0, Upper: 12.0)
Discount: 9 outliers (Lower: -0.15, Upper: 0.85)
Profit: 14 outliers (Lower: -6827.75, Upper: 16470.25)

EDA Findings

- The cleaned dataset contains 9,994 records and 21 columns, including both numerical and categorical variables suitable for business analysis.
- The dataset contains important numerical variables such as Sales, Quantity, Discount, and Profit, which can be used for business performance analysis.
- Sales and Profit show a positive relationship, indicating that higher sales are generally associated with higher profit, although this relationship is not uniform for every record.
- Sales vary across different time periods, showing noticeable changes in sales performance throughout the analyzed period.
- Product categories contribute differently to overall sales and profit, indicating differences in category-level business performance.

- The Discount variable shows variation across transactions and can be analyzed alongside Sales and Profit to understand its effect on profitability.
- Outlier analysis using the IQR method identified unusually high or low observations in numerical variables such as Sales, Quantity, Discount, and Profit.